Departamento de Informática

Bacharelado em Ciência da Computação

CI1212 Arquitetura de Computadores

Construindo o Processador

MIPS Monociclo e RISC-V

PE01-2020
Prof. Eduardo Todt
Versão 202007221631

Construindo o MIPS Monociclo

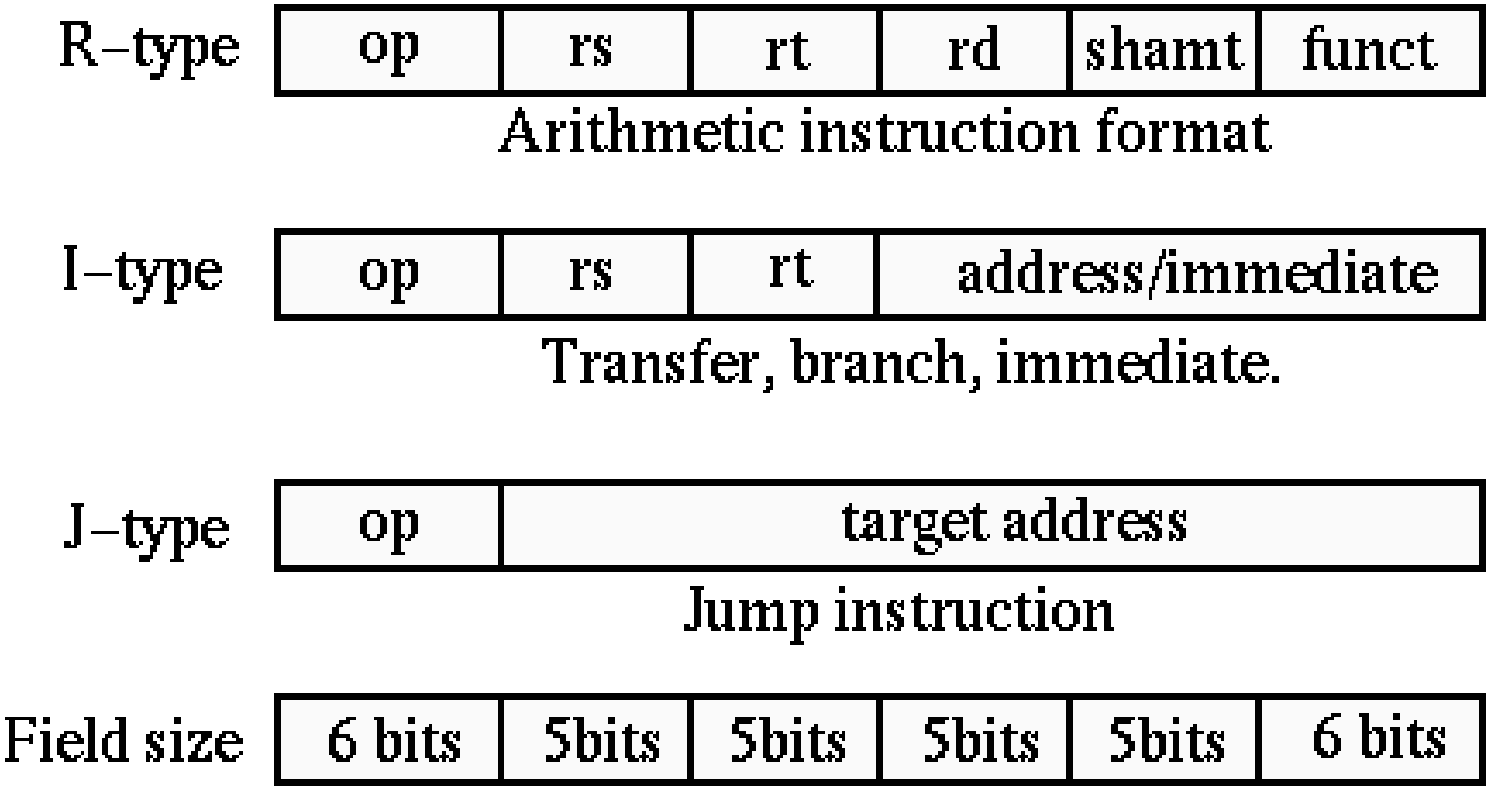
- Objetivos
  - Construir MIPS com instruções básicas:
    - lógico-aritméticas
    - controle
    - memória
  - Conhecer o RISC-V

MIPS clássico assembly

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	`$s1 = $s2 + $s3`	Three operands; data in registers
	subtract	`sub $s1,$s2,$s3`	`$s1 = $s2 – $s3`	Three operands; data in registers
Data transfer	load word	`lw $s1,100($s2)`	`$s1` = Memory[`$s2` + 100]	Data from memory to register
	store word	`sw $s1,100($s2)`	Memory[`$s2` + 100] = `$s1`	Data from register to memory
Logical	and	`and $s1,$s2,$s3`	`$s1 = $s2 & $s3`	Three reg. operands; bit-by-bit AND
	or	`or $s1,$s2,$s3`	`$s1 = $s2 | $s3`	Three reg. operands; bit-by-bit OR
	nor	`nor $s1,$s2,$s3`	`$s1 = ~ ($s2 |$s3)`	Three reg. operands; bit-by-bit NOR
	and immediate	`andi $s1,$s2,100`	`$s1 = $s2 & 100`	Bit-by-bit AND reg with constant
	or immediate	`ori $s1,$s2,100`	`$s1 = $s2 | 100`	Bit-by-bit OR reg with constant
	shift left logical	`sll $s1,$s2,10`	`$s1 = $s2 << 10`	Shift left by constant
	shift right logical	`srl $$s1,$s2,10`	`$s1 = $s2 >> 10`	Shift right by constant
Conditional branch	branch on equal	`beq $s1,$s2,L`	if (`$s1 == $s2`) go to `L`	Equal test and branch
	branch on not equal	`bne $s1,$s2,L`	if (`$s1 != $s2`) go to `L`	Not equal test and branch
	set on less than	`slt $s1,$s2,$s3`	if (`$s2 < $s3`) `$s1` = 1; else `$s1` = 0	Compare less than; used with `beq`, `bne`
	set on less than immediate	`slt $s1,$s2,100`	if (`$s2 < 100`) `$s1` = 1; else `$s1` = 0	Compare less than immediate; used with `beq`, `bne`
Unconditional jump	jump	`j L`	go to `L`	Jump to target address

MIPS clássico codificação instruções



Visão geral da CPU

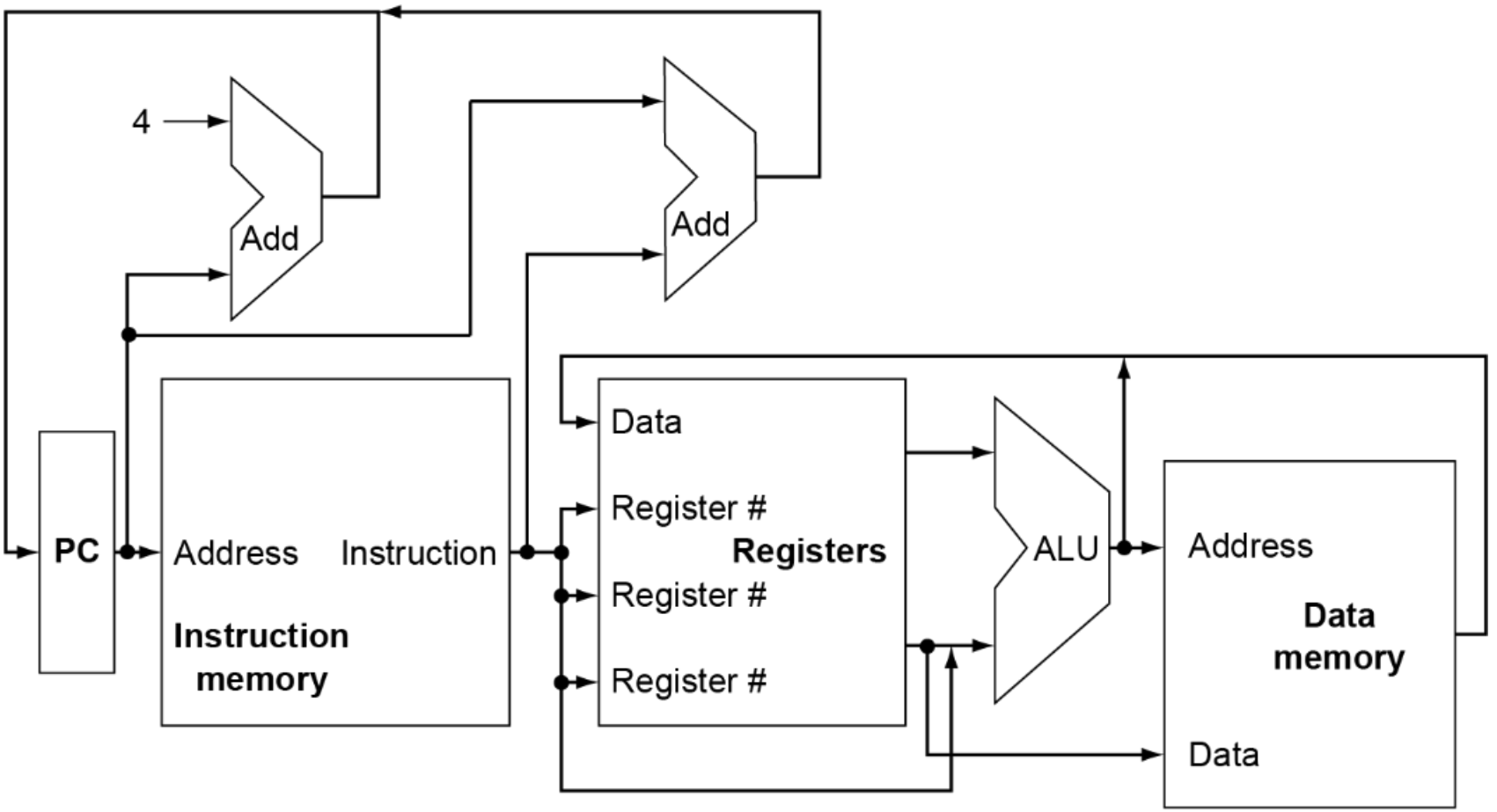
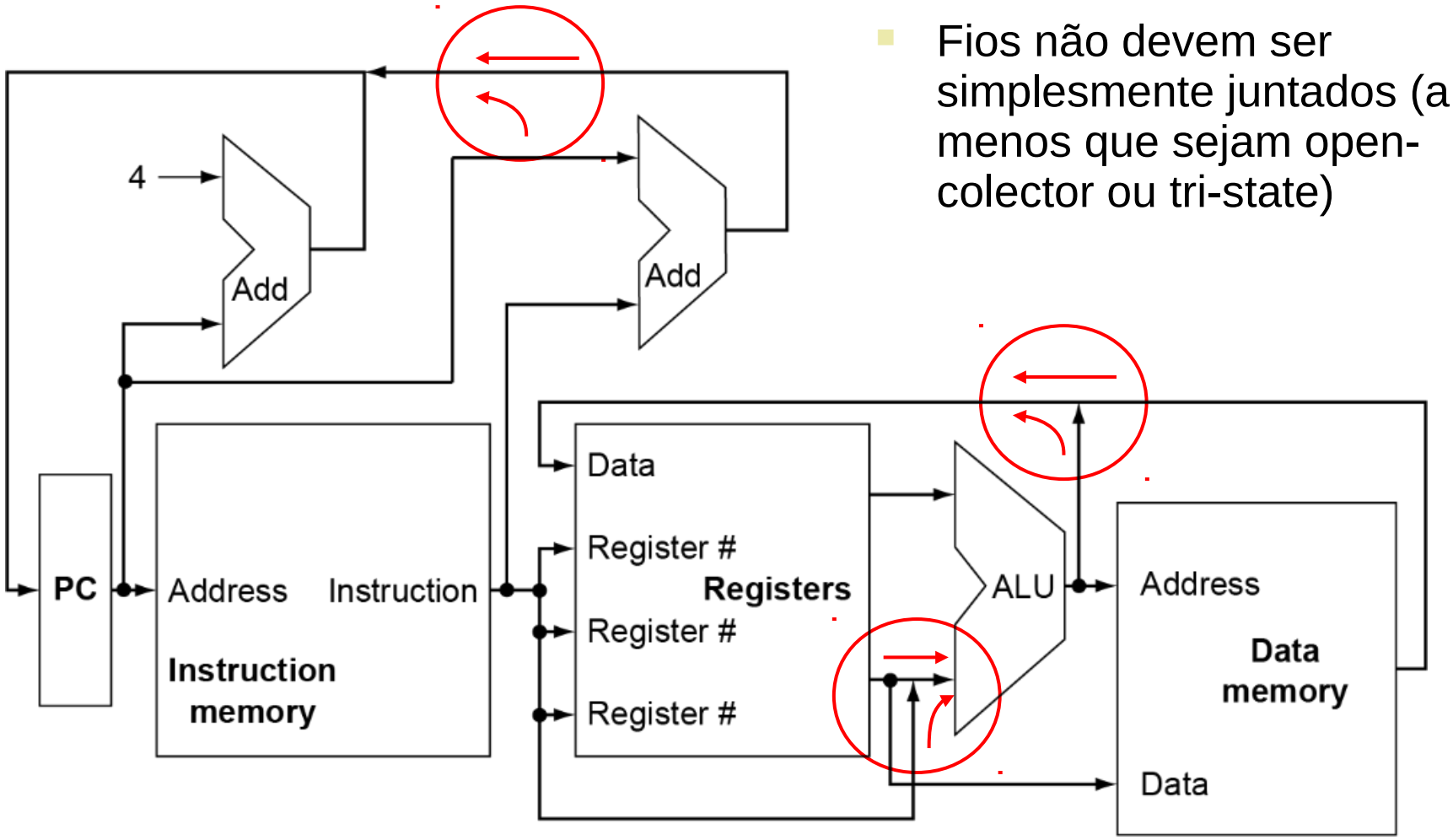


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Multiplexadores

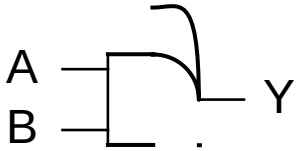


- Fios não devem ser simplesmente juntados (a menos que sejam open-colector ou tri-state)

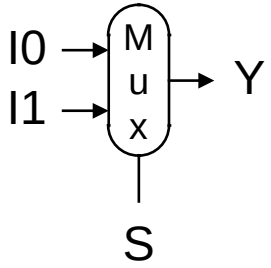
Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Elementos combinacionais

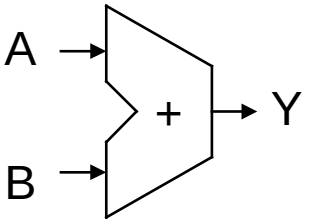
- AND-gate
  - Y = A & B



- Multiplexer
  - Y = S ? I1 : I0



- Adder
  - Y = A + B



- Arithmetic/Logic Unit
  - Y = F(A, B)

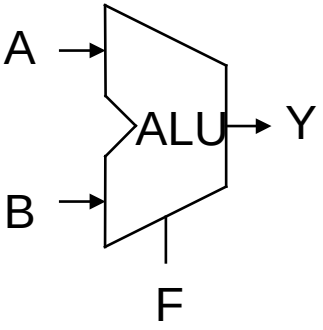
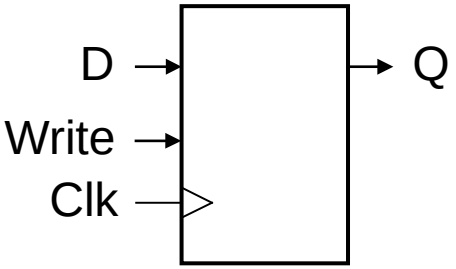


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Elementos sequenciais



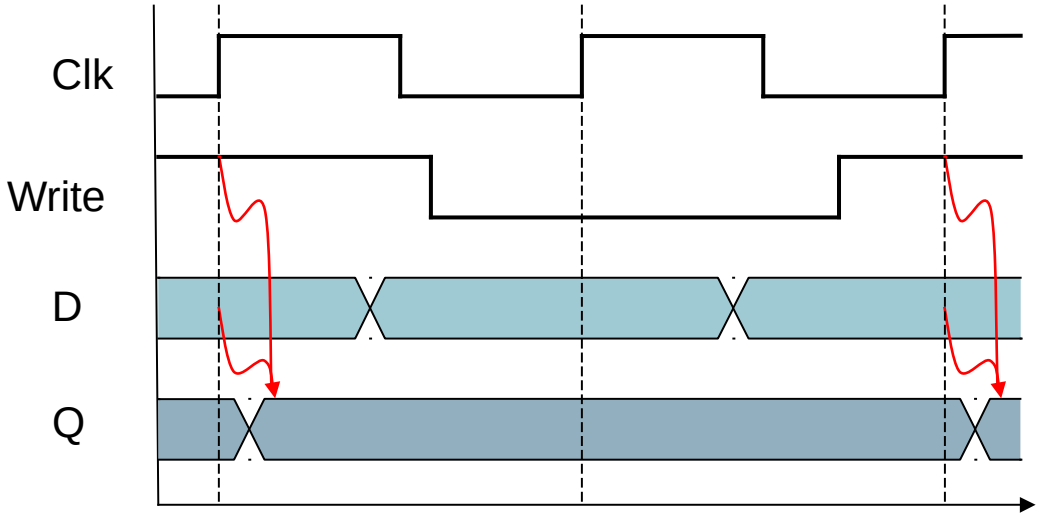
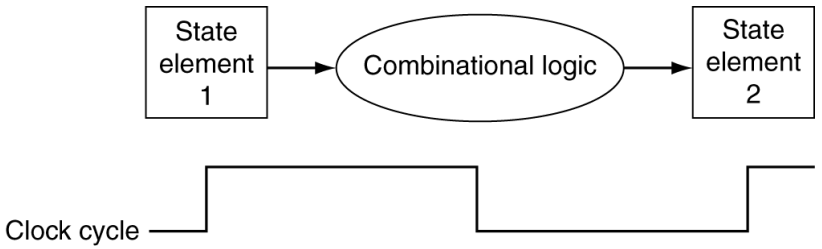


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Circuitos sequenciais

Lógica combinacional transforma dados durante ciclos de clock

- Entre bordas de clock
- Entradas dos elementos sequenciais, saídas aos elementos sequenciais
- Maior atraso determina período de clock



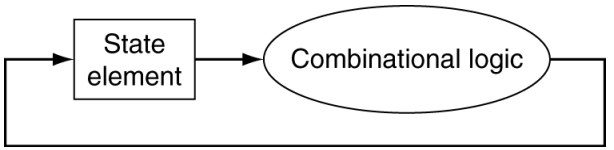


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Revisando o fetch

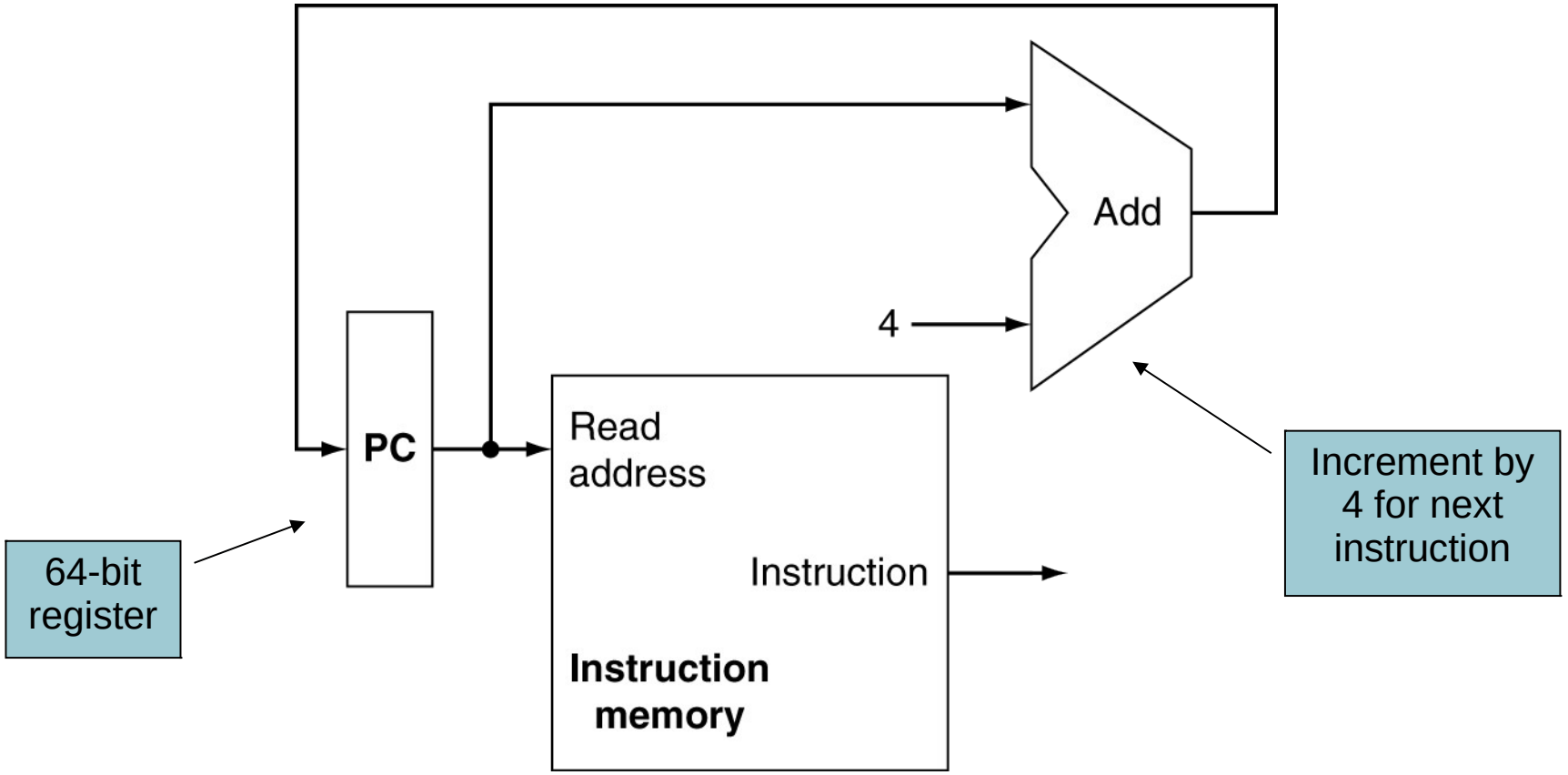
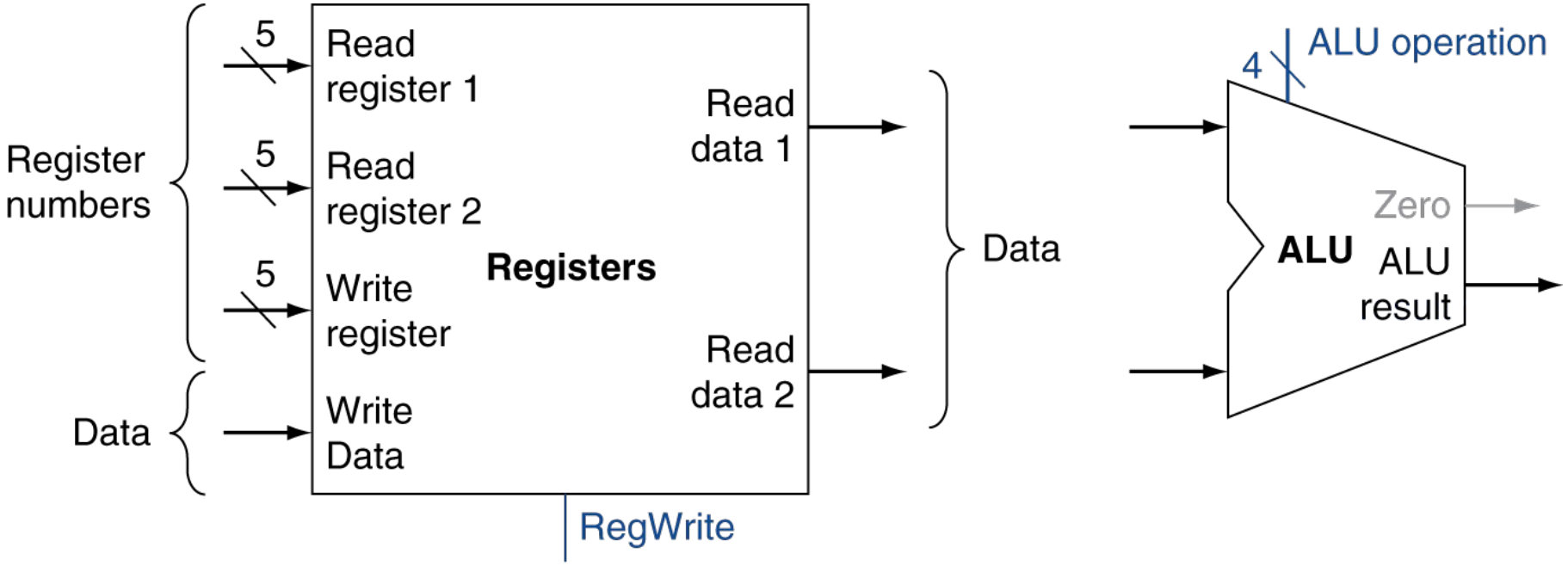


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Instruções tipo R

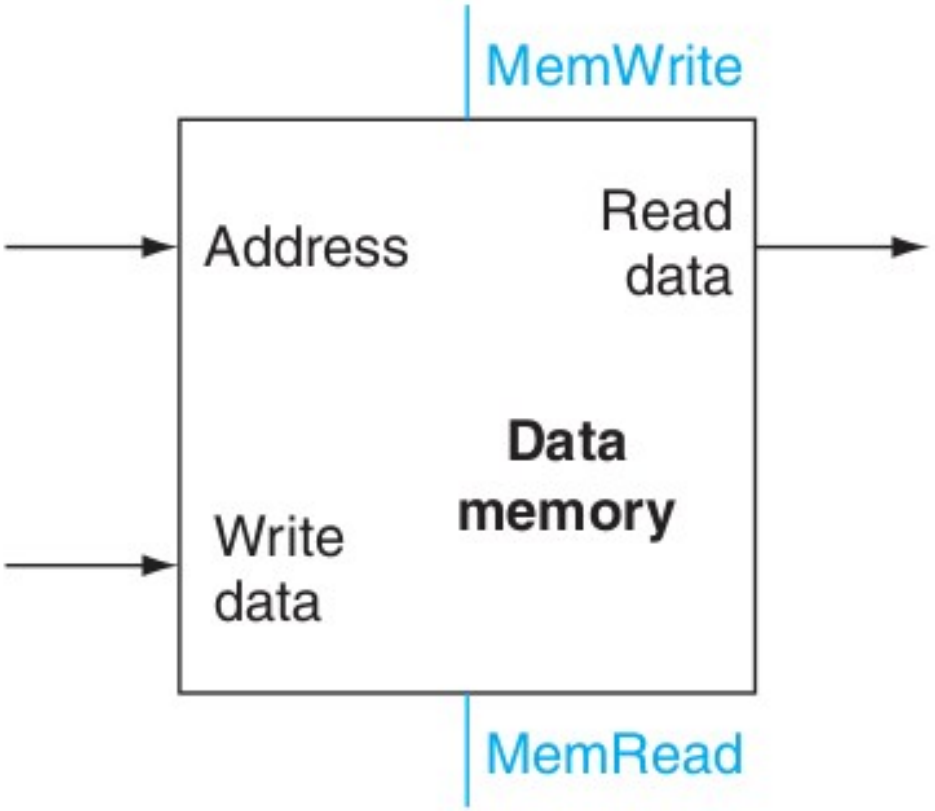


a. Registers

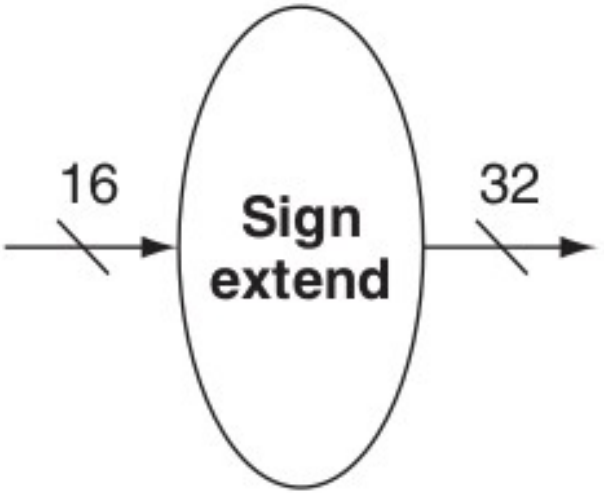
b. ALU

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Instruções load/store



a. Data memory unit



b. Sign-extension unit

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

Instruções de desvio

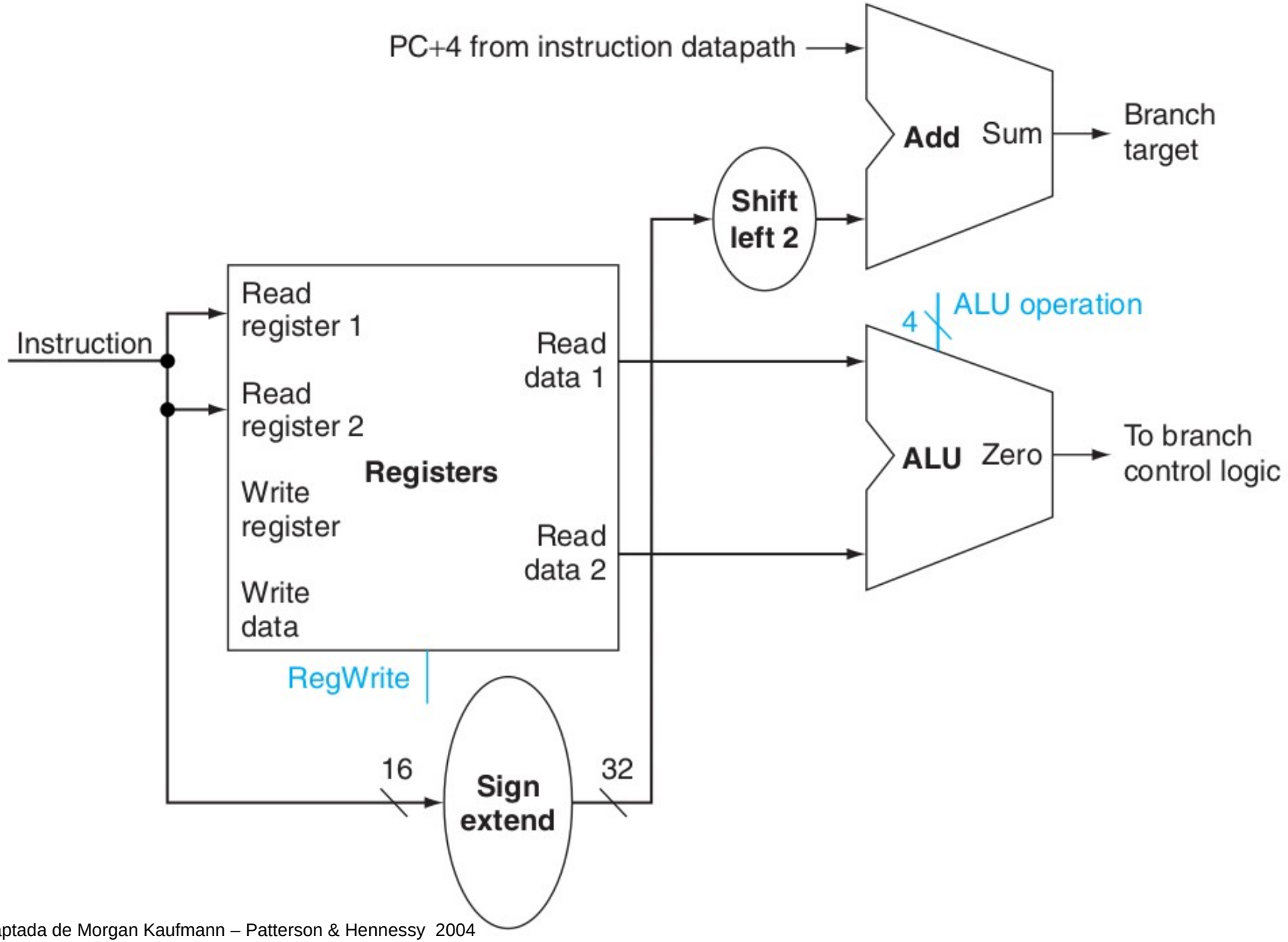


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

Datapath R e Load/Store

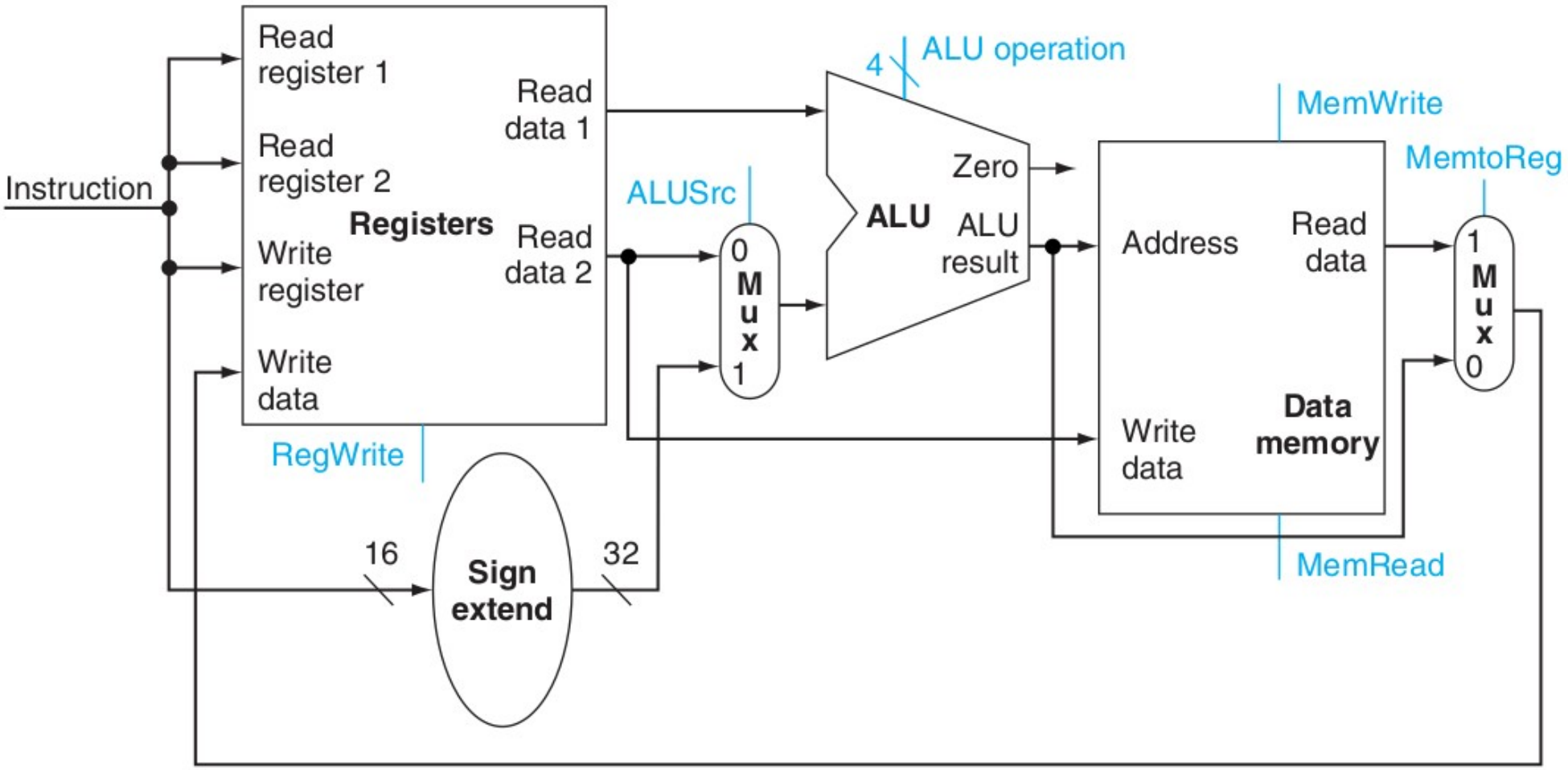


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

Datapath R, lw/sw, beq

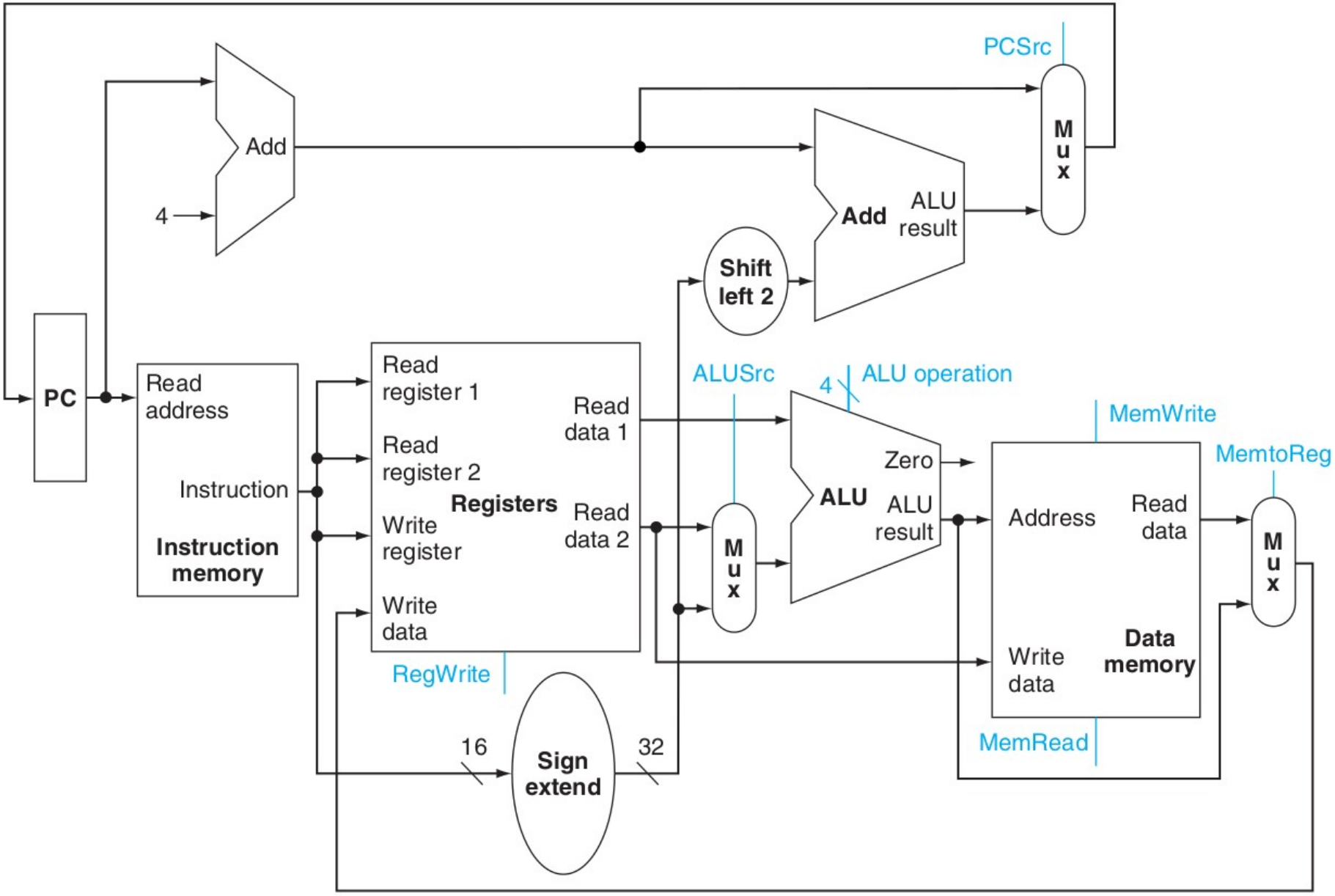
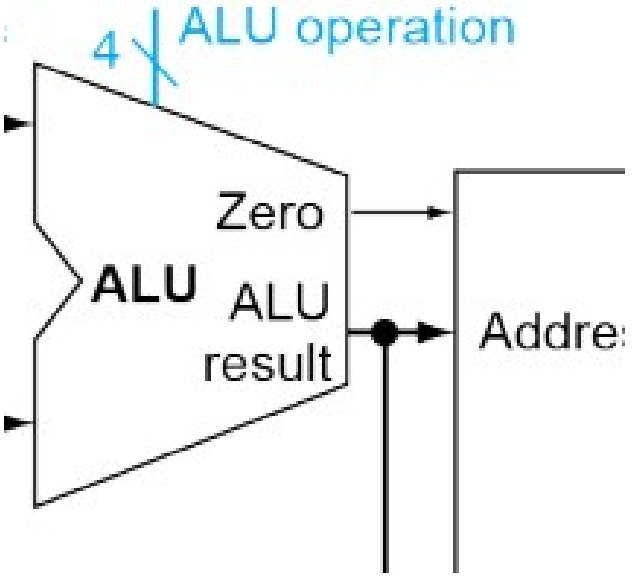


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

Controle da ULA

ULA utilizada para

- Load/Store: F = add
- Branch: F = subtract
- R-type: F depende do opcode

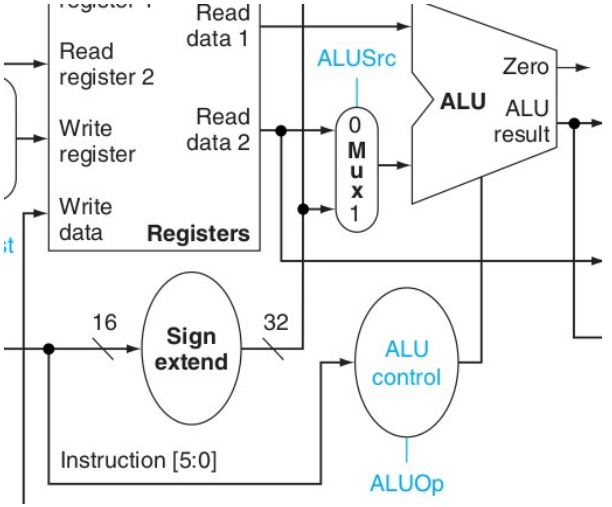


ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

Controle da ULA

Considere 2-bit ALUOp derivado do opcode
Outra lógica combinacional determina controle da ULA



Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	and	0000
R-type	10	OR	100101	or	0001
R-type	10	set on less than	101010	set on less than	0111

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

MIPS clássico Datapath com controle

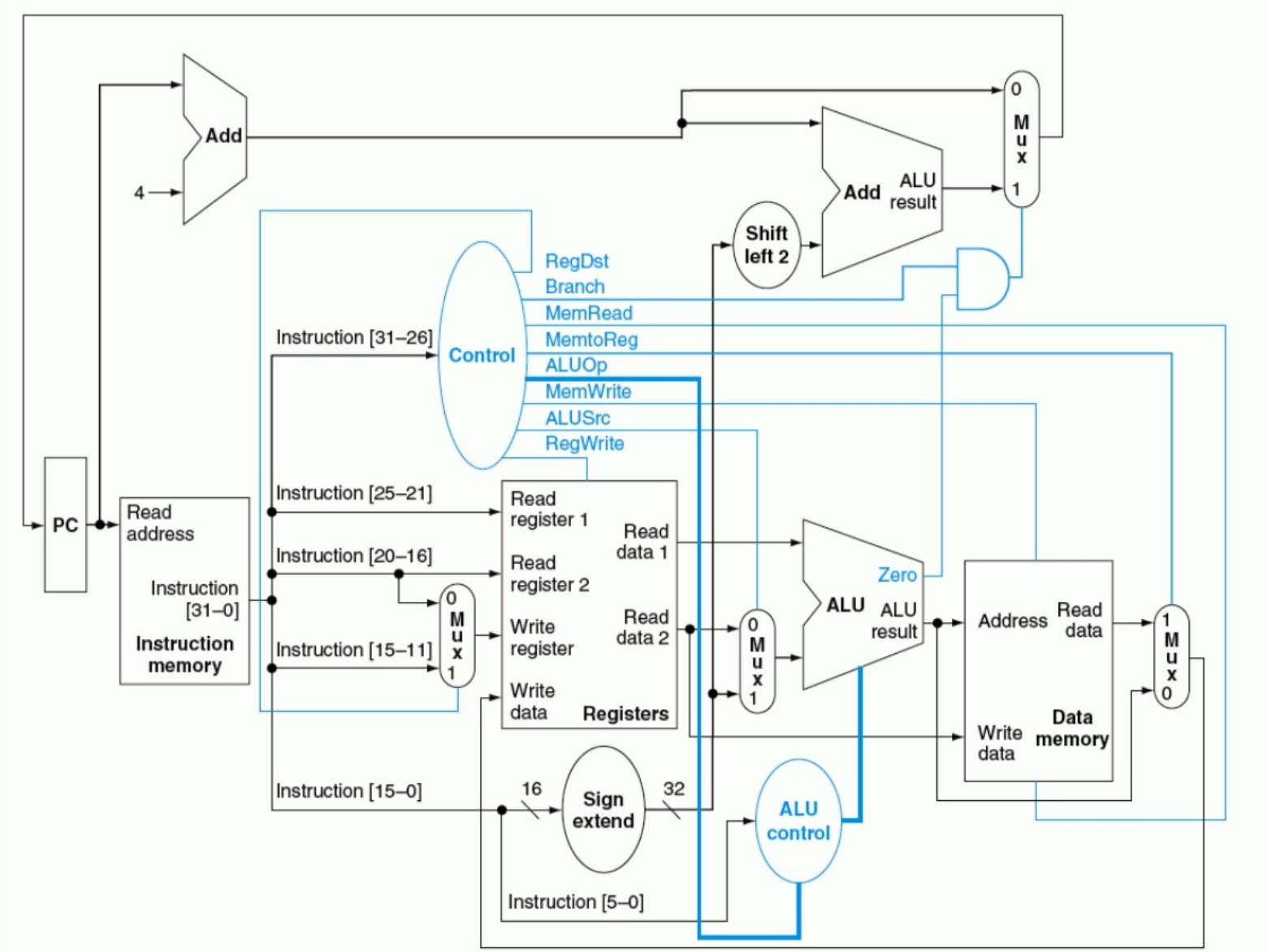


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

MIPS clássico Datapath com controle e Jump

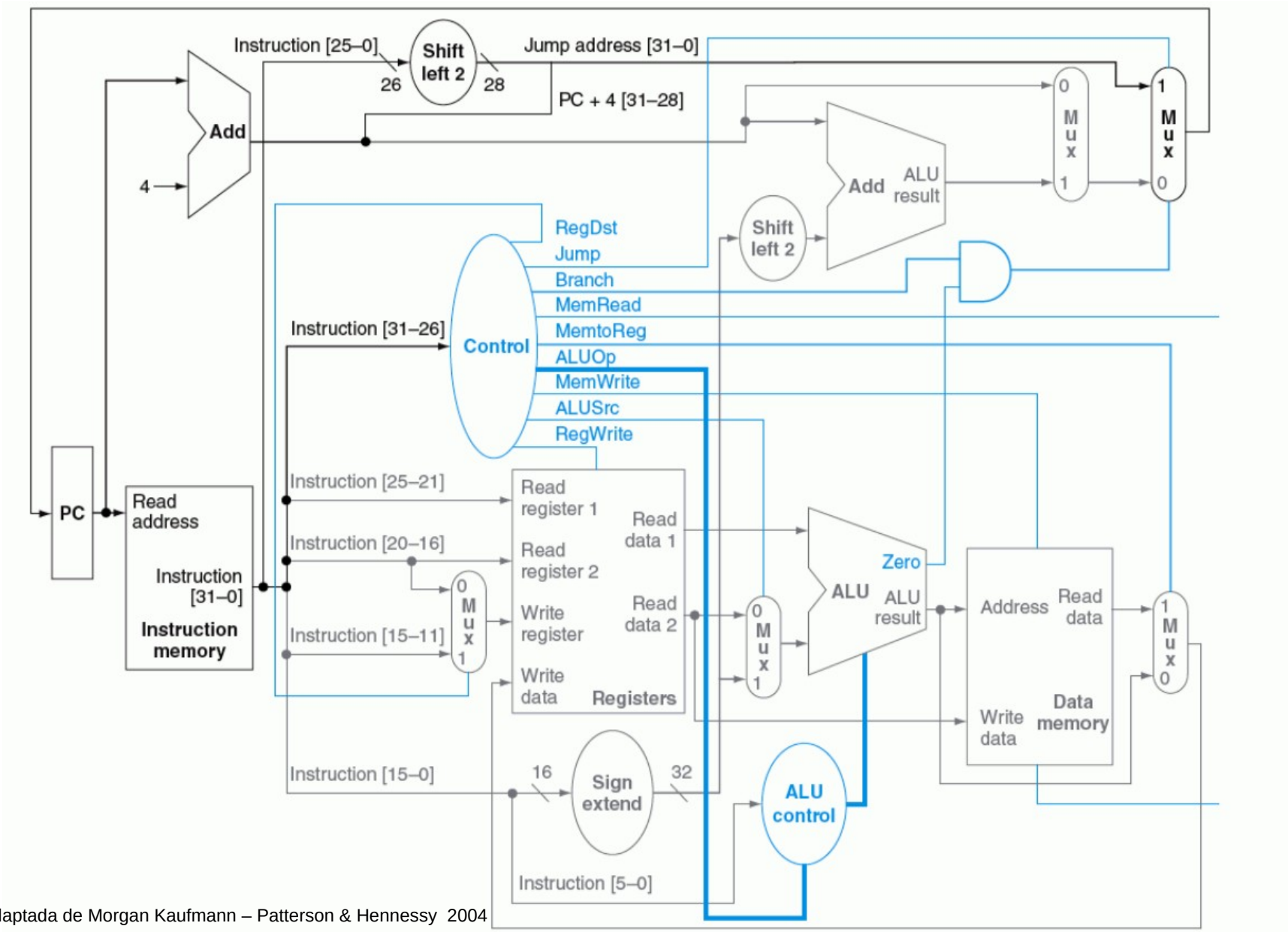
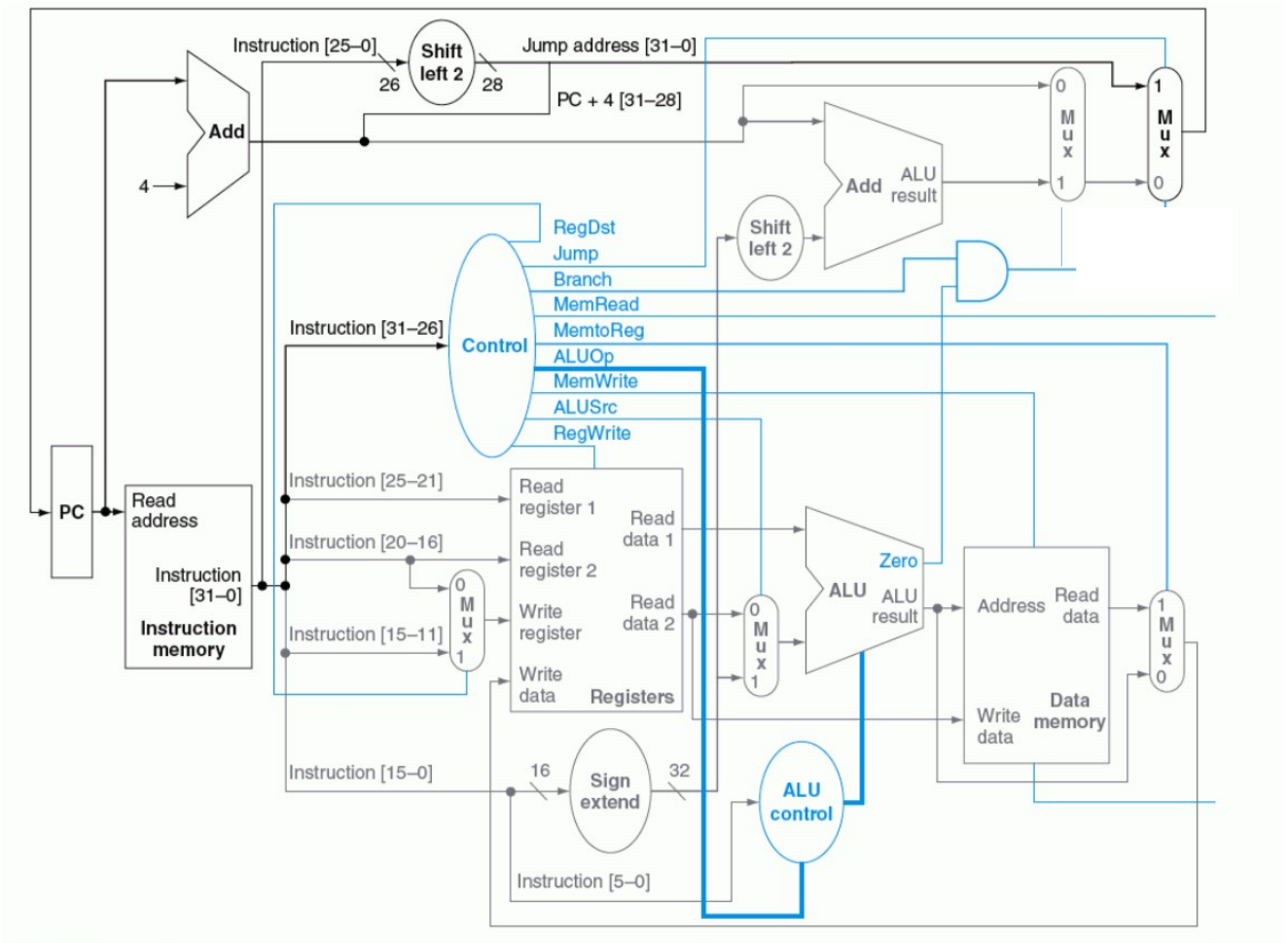


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

Unidade de controle



Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

Questões de desempenho

Maior caminho de atraso define período clock

- Caminho crítico: load
- Memória instruções → registradores → ULA → memória dados → registradores

Não é possível período variável conforme instrução

Viola princípio “Fazer caso comum rápido”

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

MIPS clássico
Tempo para R type

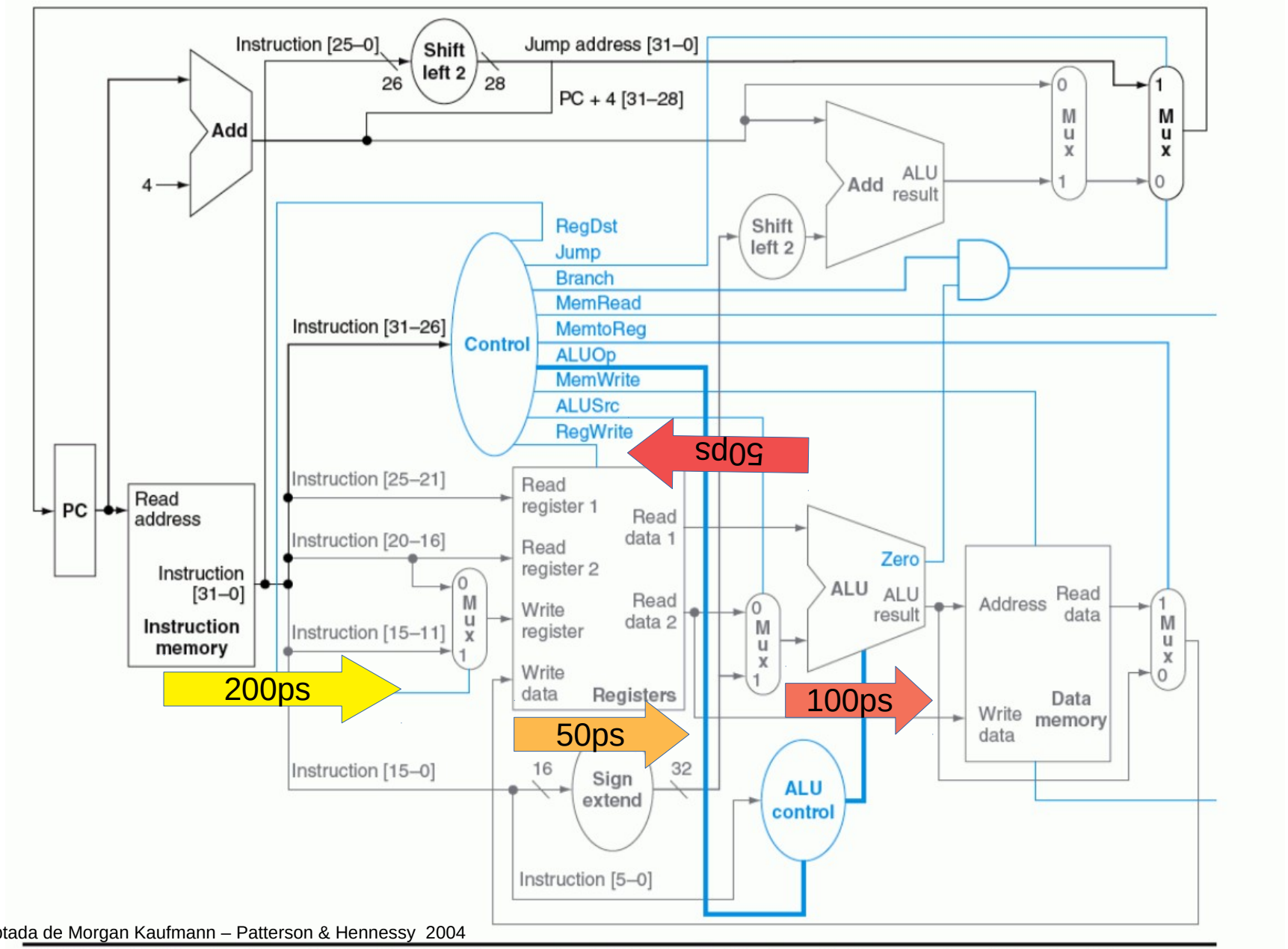


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

MIPS clássico
Tempo para beq

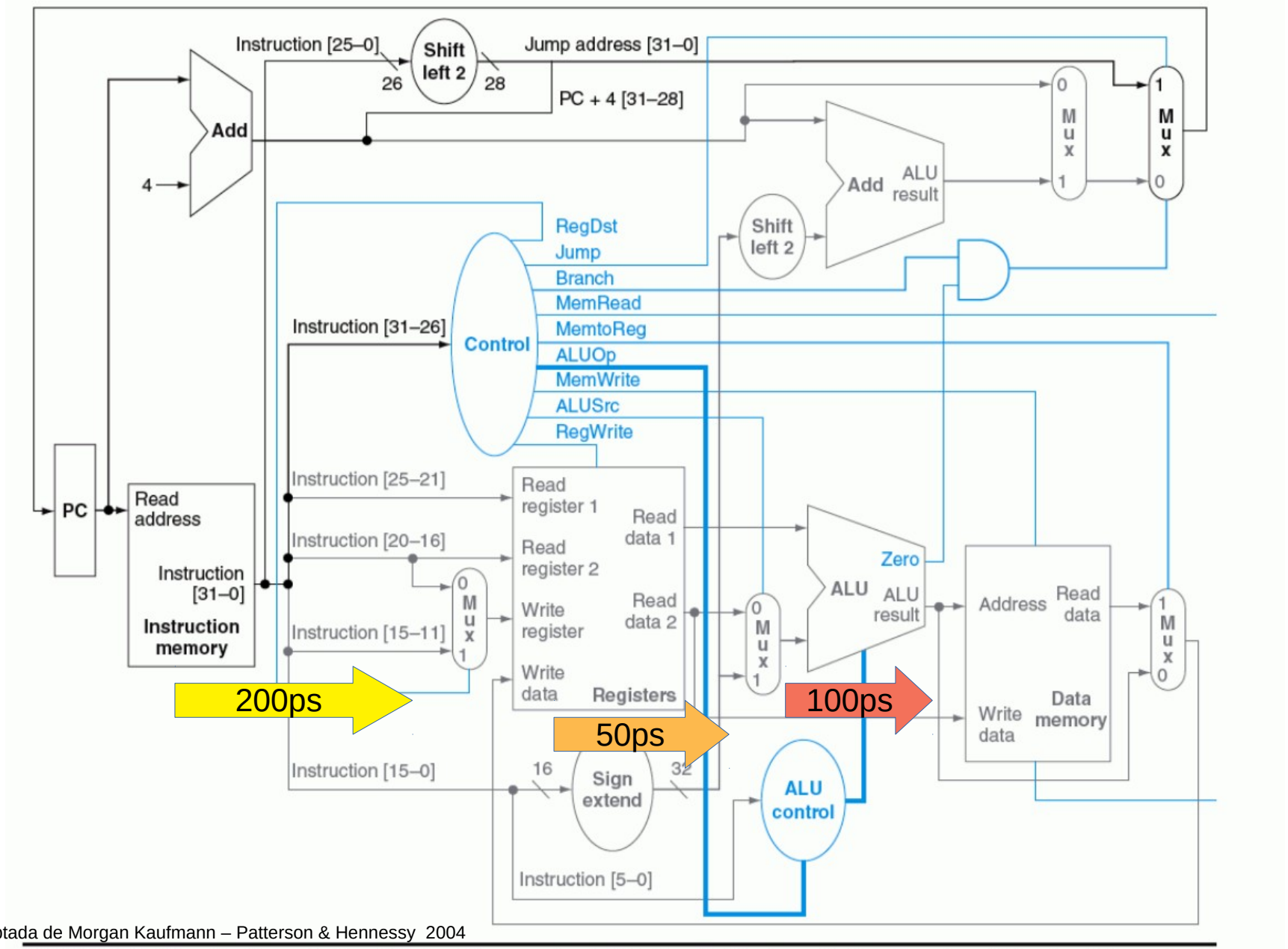


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

MIPS clássico
Tempo para sw

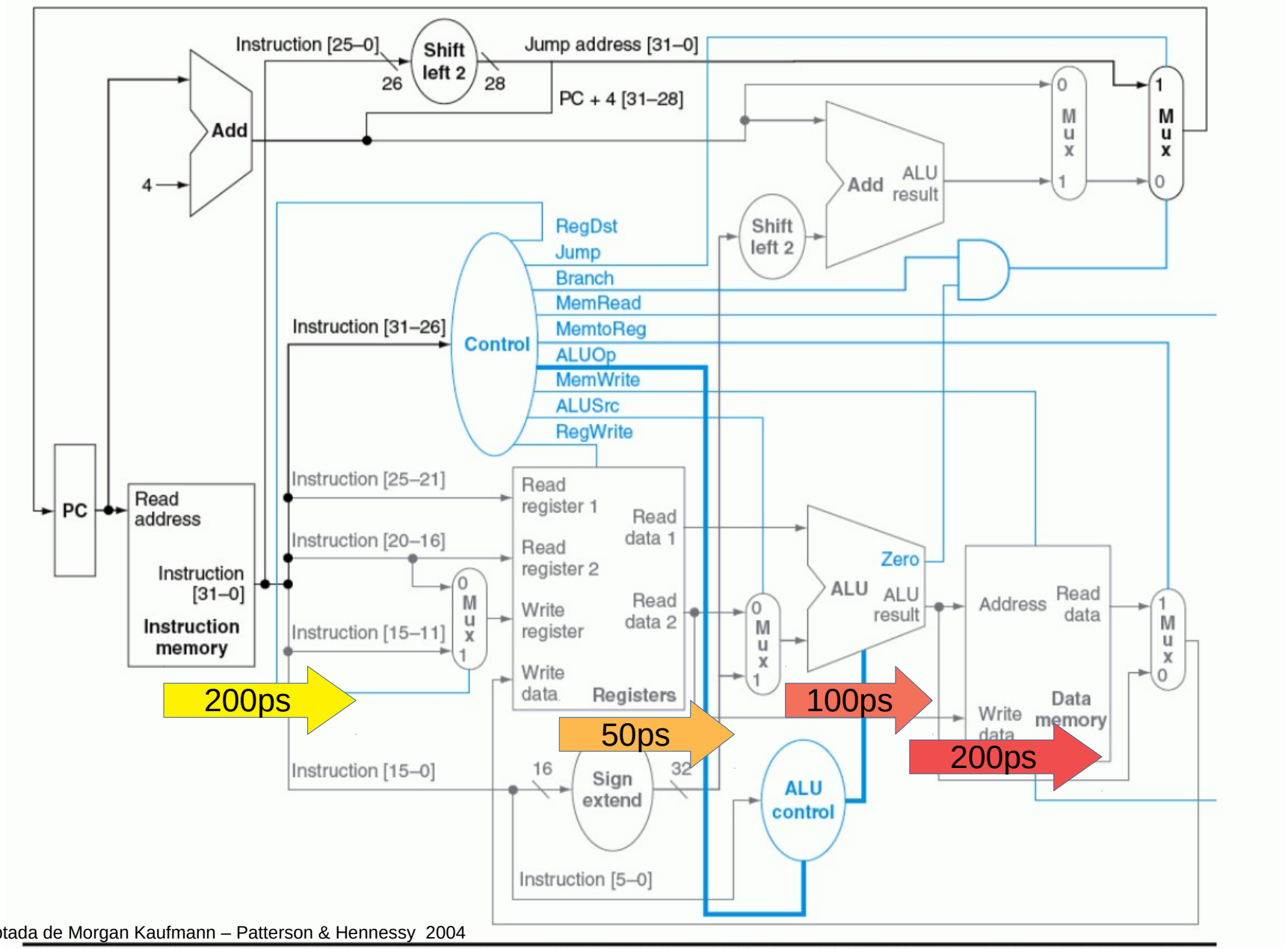


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

MIPS clássico
Tempo para lw

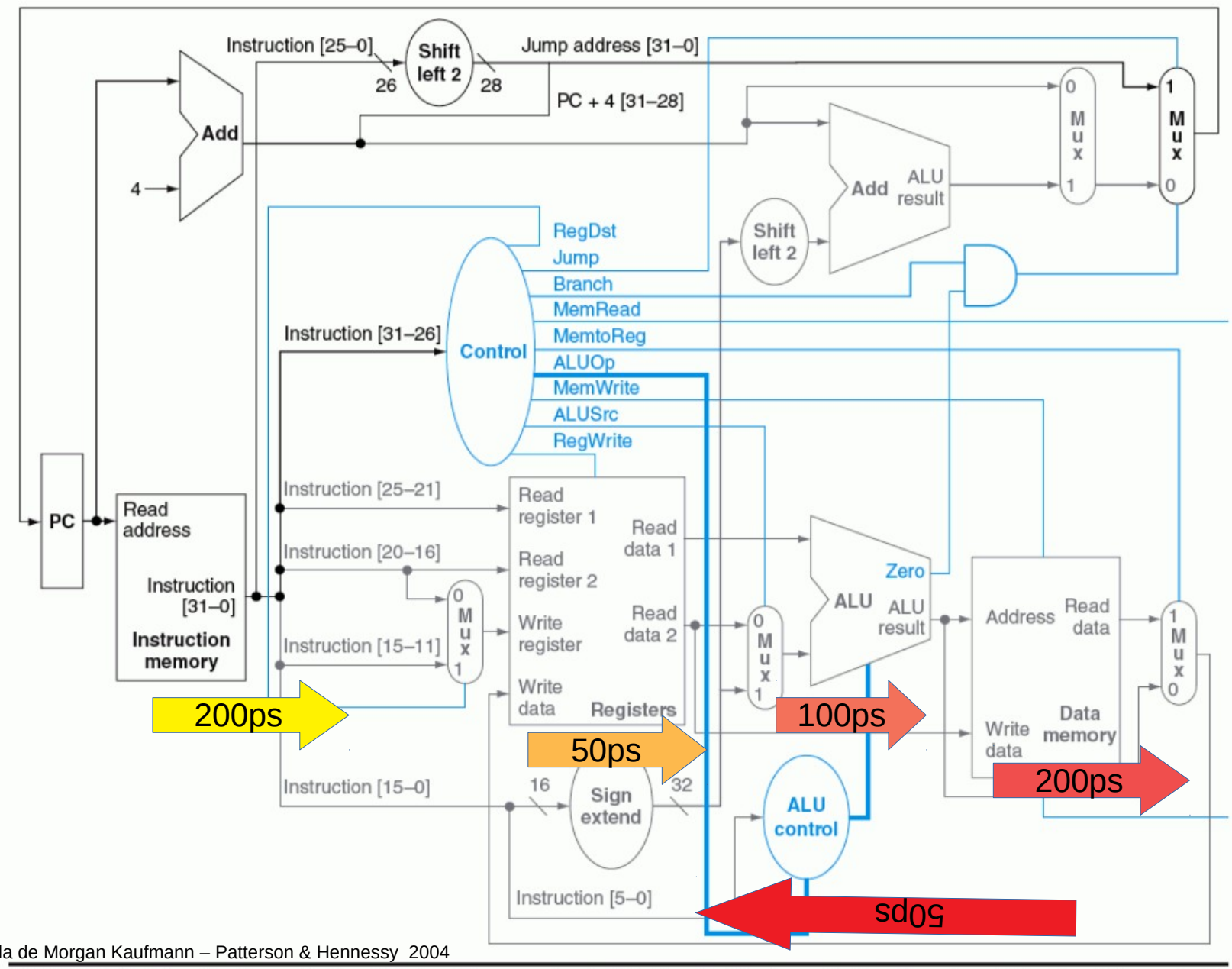


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

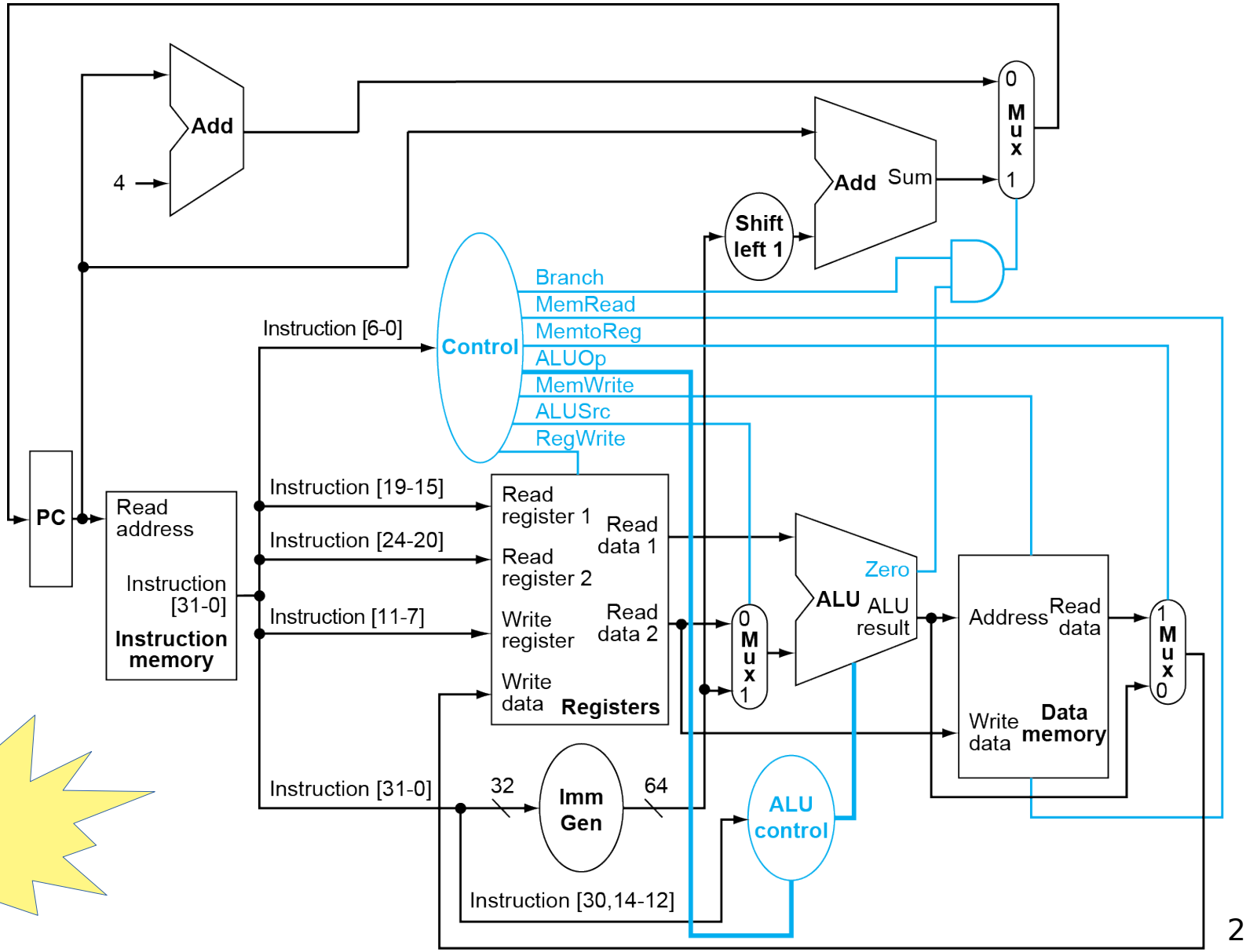
MIPS → RISC-V

Computer Architecture: A Quantitative Approach 1ª ed. em 1990 autores criaram o processador MIPS, com ISA proprietário.

Em 2010 ISA tornou-se aberto, com o RISC-V

- Formato de instruções flexível com instruções de 32 e de 64 bits, palavras de 32, 64 e 128 bits, redução do uso de multiplexadores, FPU IEEE 754

Datapath com controle

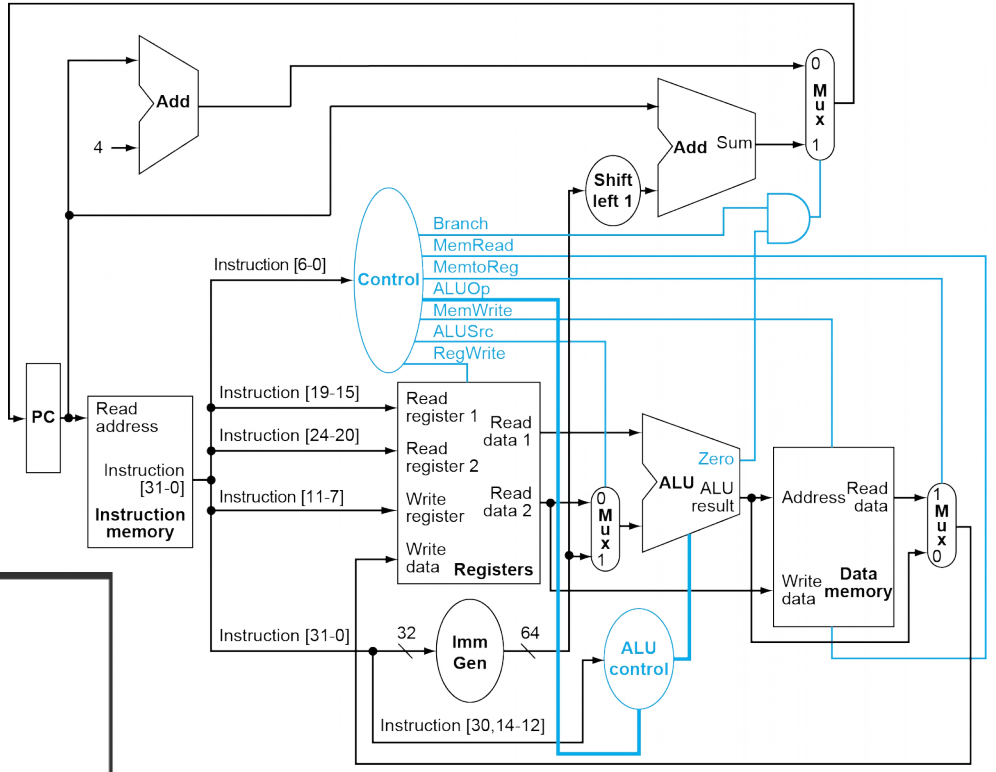


RISC-V

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Unidade de controle

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode



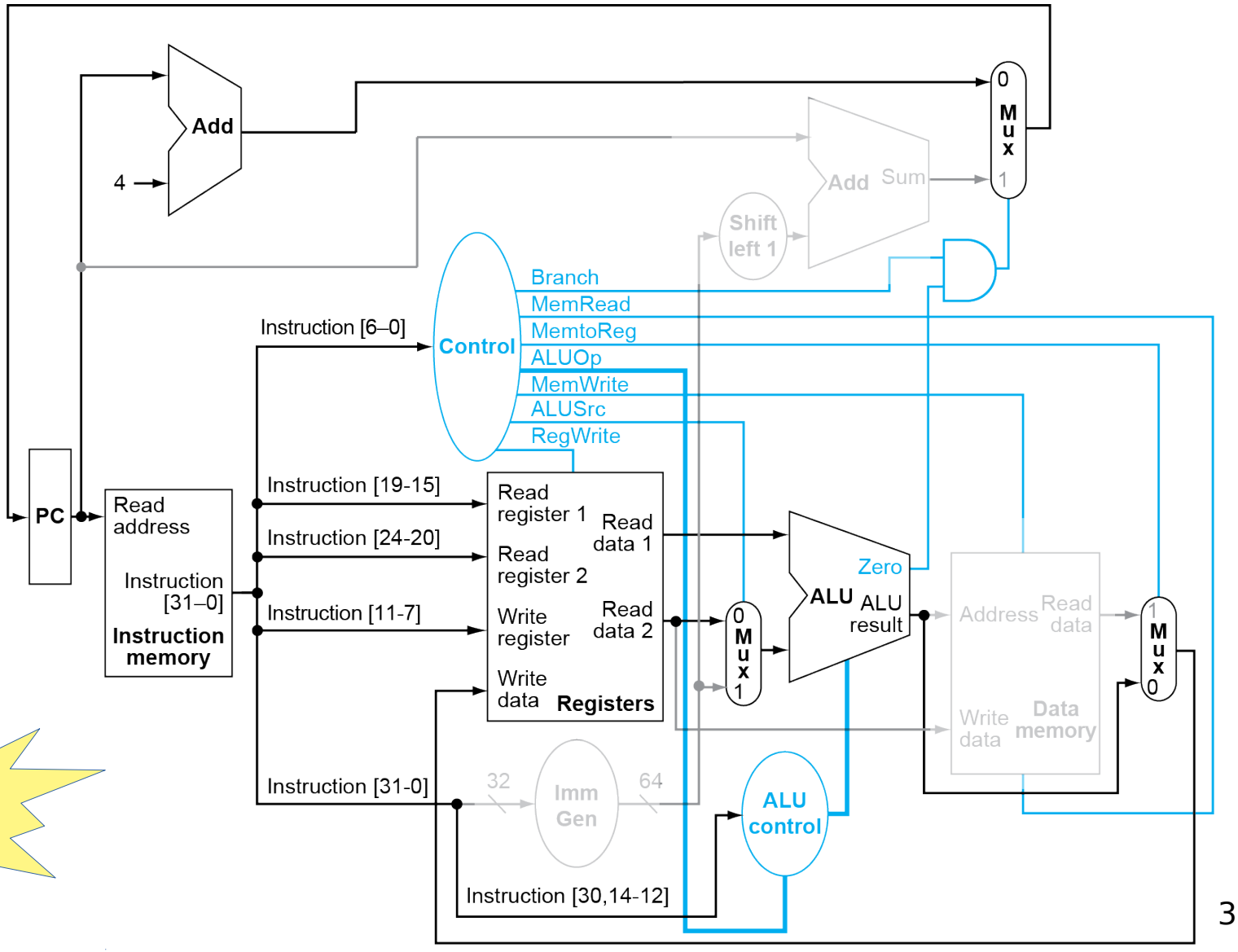
ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Instruction opcode	ALUOp	operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

RISC-V

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

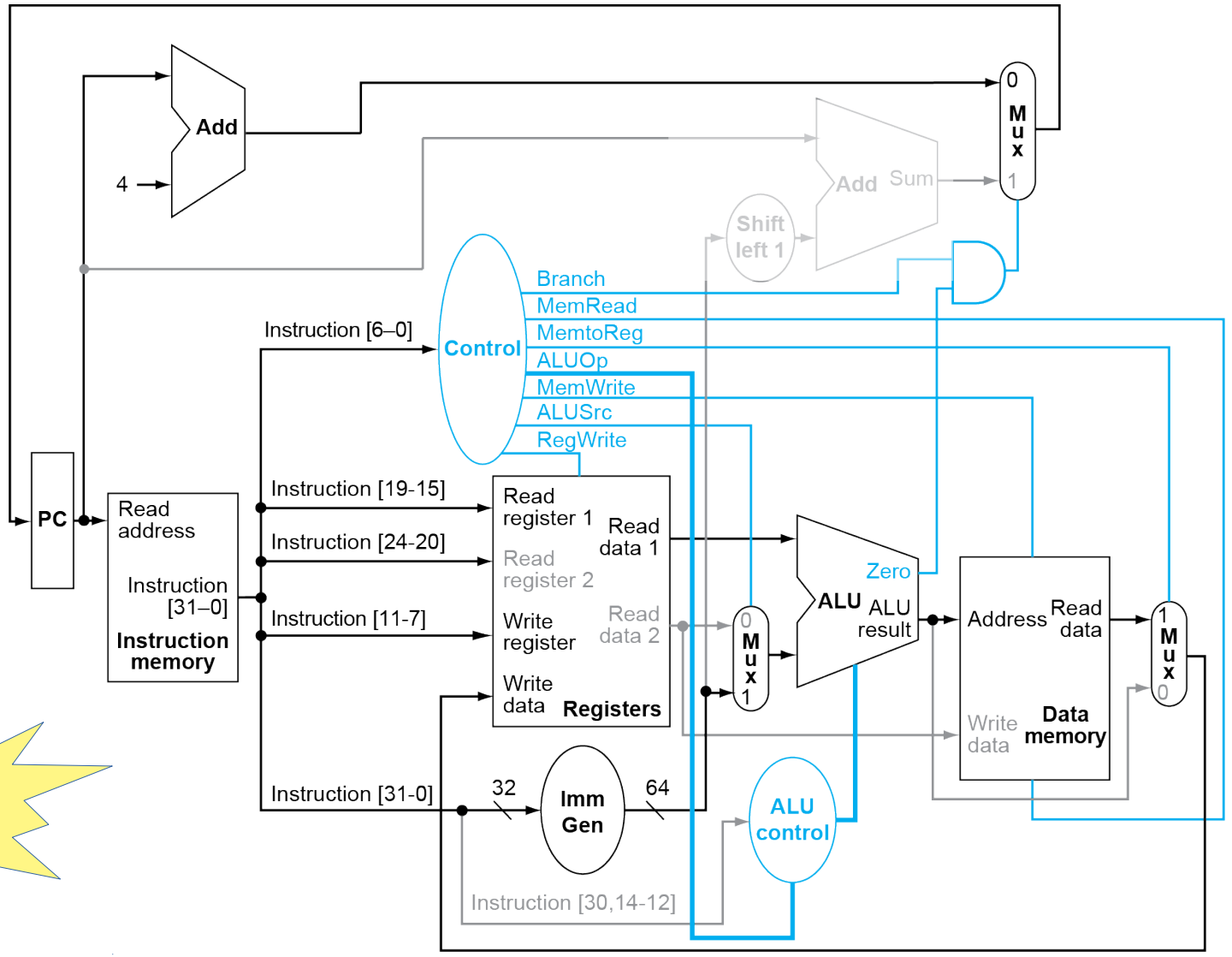
Instrução R
ex: add $1,$2,$3



RISC-V

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

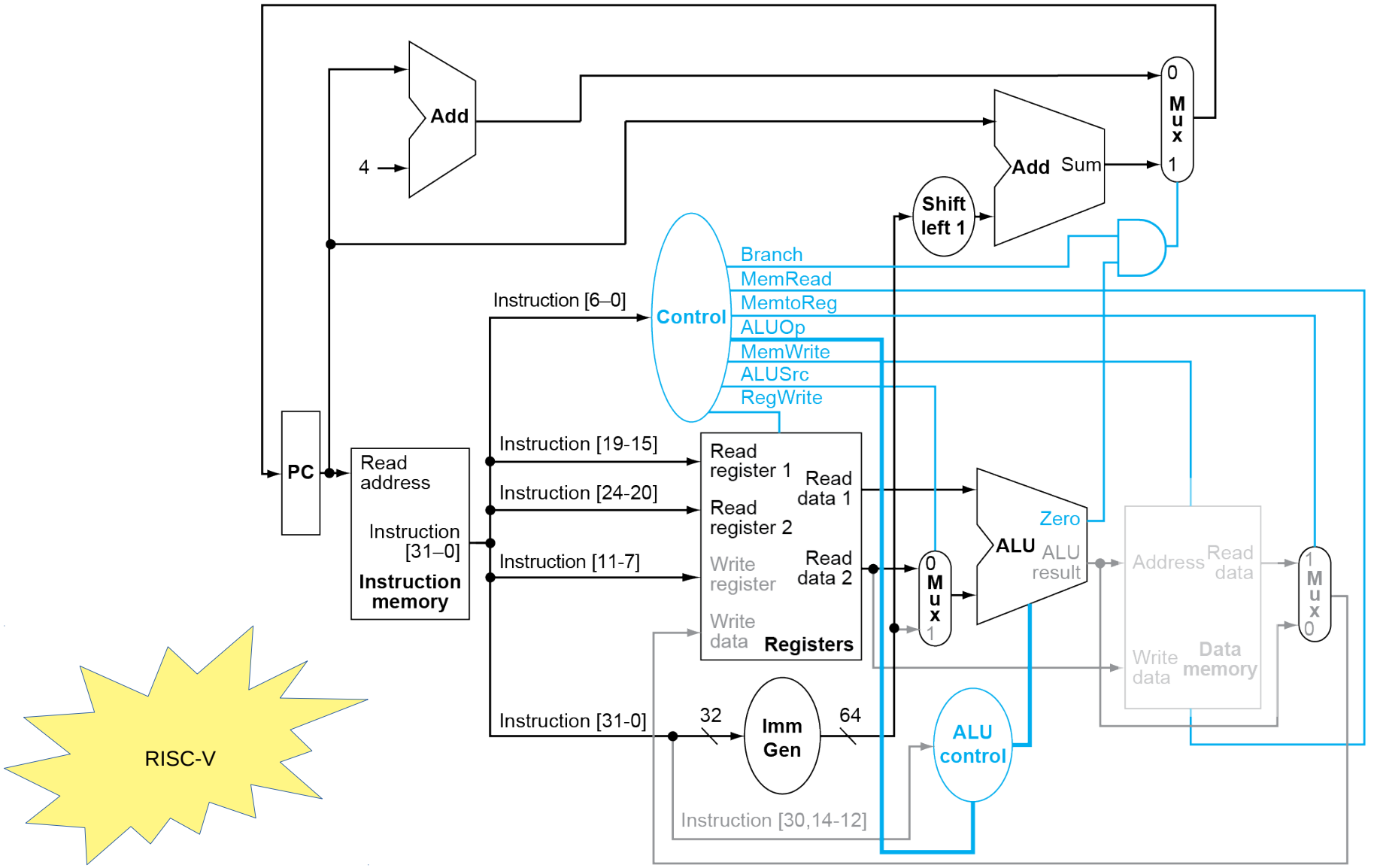
Instrução Load
ex: ld $1, 16($2)



RISC-V

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Instrução Beq
ex: beq $1,$2, 0x0100



RISC-V

Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2018

Créditos

Figuras de Patterson&Hennessy 2018, 2004 – Morgan Kaufmann

CI1212 Arquitetura de Computadores

Prof. Eduardo Todt

todt@inf.ufpr.br

Período Especial 2020 01